



Software Requirement in Iterative SDLC Model

Olatunji J. Okesola¹(✉), Ayodele A. Adebisi^{2,3}, Ayoade A. Owoade⁴,
Oyetunde Adeaga⁵, Oluseyi Adeyemi⁵, and Isaac Odun-Ayo³

¹ Department of Mathematics and Computer Sciences,
First Technical University, Ibadan, Nigeria
olatunjiokesola@tech-u.edu.ng

² Department of Computer and Information Sciences,
Land Mark University, Omu-Aran, Nigeria
ayo.adebiyi@lmu.edu.ng

³ Department of Computer and Information Sciences, Covenant University, Ota, Nigeria
{ayo.adebiyi, isaac.odun-ayo}@covenantuniversity.edu.ng

⁴ Department of Computer Science, Tai Solarin University, Ijebu-Ode, Nigeria
owoadea@tasued.edu.ng

⁵ Department of Mechanical Engineering, First Technical University, Ibadan, Nigeria
{oyeadeaga, oaadeyemi}@tech-u.edu.ng

Abstract. Software development life cycle (SDLC) is critical and remains the most significant part of software development in requirement engineering. There exists several SDLC each designed for specific needs to accommodate available resources and skills; they are compatible with certain programming languages and toolkits, and are of immense benefits to the software engineers. Despite its popularity and strengths over major weaknesses of other models, Iterative model is less acceptable by programmers and not widely in use in industry. This study examines the concepts of Iterative models and presents its real time examples to make the SDLC more understood to software developers. The practical application of the model is exemplified, motivation for its use are highlighted, and the requirement as well as requirement activities in each phase are tabulated. This will raise users' awareness on the Agile Software Development Model and attract software developers to appreciate the Iterative Model and make a better use.

Keywords: Iterative model · Practical application · Requirement activities · Requirement engineering · Software development life cycle · Software engineering

1 Introduction

Software Development Lifecycle (SDLC) is the process of developing information system with proper analysis, design, implementation and maintenance. It crucial in system development as it breaks down the entire life cycle of software development thus making it easier to evaluate each part of software development while providing some levels of comforts for programmers to work concurrently on each phase.

A typical SDLC as illustrated in Fig. 1 is an abstract representation of a software process and defines the phases, steps, activities, methods, tools as well as expected deliverables, of a software development project [1]. It is a process that guides a software team through a set of framework activities which are organized into a process flow that may be linear, incremental, or evolutionary [2]. The Phases in a Software lifecycle are well-defined and are aimed at helping the developer deliver an elite system which fits or surpasses the stakeholder's prospects, established on stakeholder's requirements [3].

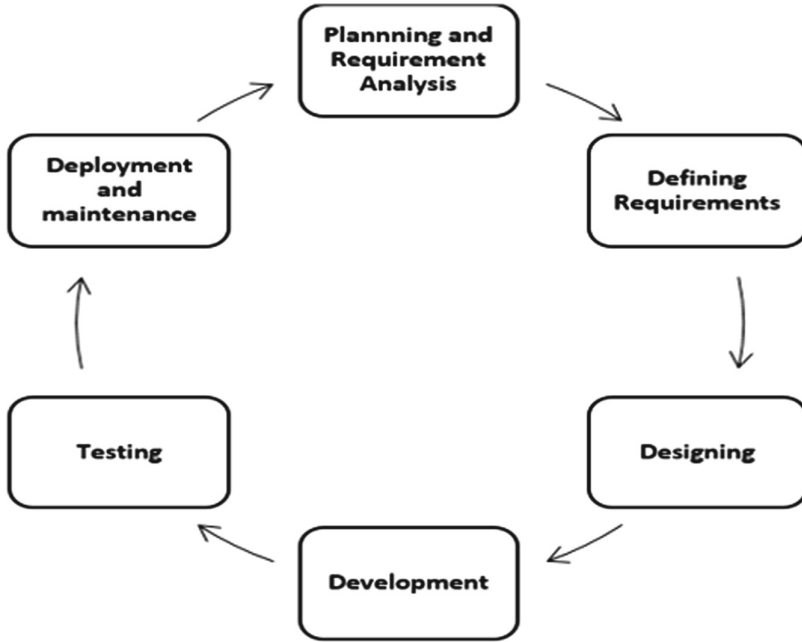


Fig. 1. A typical SDLC [3].

Different approaches to software development may carry out these stages in different orders, or devote more or less time to different stages; the details of the documentation produced at each stage of software development may also be at diverge [4]. Hence, emphasis are not being placed on the timing and order in which the activities takes place. However, depending on development approaches deployed, a stage may attract more or less attention, and may not necessarily come in a particular order. The details of the documentations at each stage may also be at diverge [4]. Hence, emphasis are not being placed on the timing and order in which the activities takes place.

Activities in SDLC occur in many ways and at a rigid sequence, with activities running concurrently, or one complete before the other begins. This gives rise to several SDLC models including Waterfall Model, Iterative Model, Spiral Model, V-Model, and Big Bang Model. In this study, only iterative model is considered extensively and its application in the real life discussed.

2 The Iterative Model

Iterative development is a rework scheduling strategy in which time is set aside to revise and improve parts of the system [5]. In this case, feature code is designed, developed and tested in repeated cycles. A detailed requirement is not required to start with as improvement starts by indicating and executing parts of the software. This procedure is then rehashed to create another rendition of the software toward the finish of every cycle of the model.

Iterative development model (Fig. 2) separates the product advancement of an expansive application into little pieces. It is therefore regarded as the process of breaking down the software development of a large application into smaller chunks.

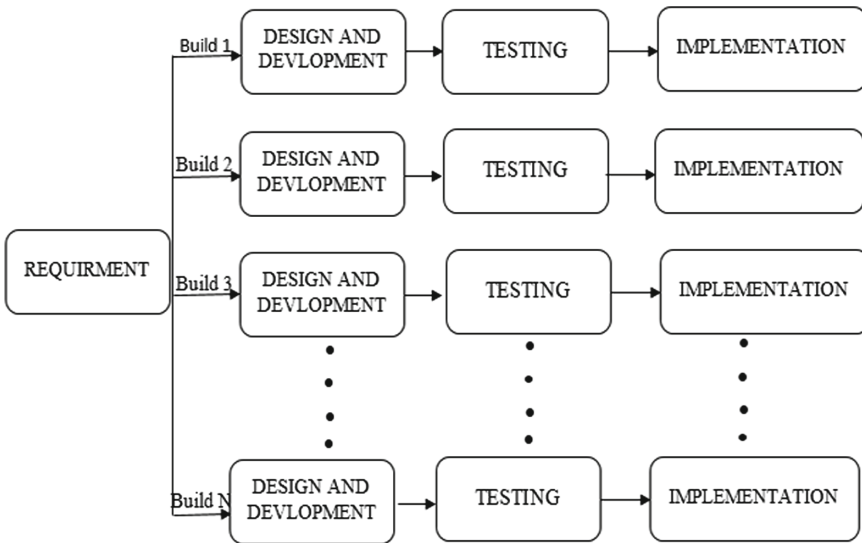


Fig. 2. An Iterative model (adapted from Motea, 2015)

2.1 The Advantages

A significant feature of the iterative model is that there is a working model of the entire system at the start of the development exercise, making it easier to promptly discover configuration errors towards taking restorative measures in planning cost. Some other notable strength of the model include the following:

Inborn Variation: It is fairly clear that most software development life cycles will incorporate some type of variations, demonstrating the deployment phase of the product at a specific stage. Nonetheless, the iterative model makes this considerably less demanding by guaranteeing that there is a more current emphases are incrementally enhanced renditions of past cycles [7].

Fast Turnaround: While it might appear like each phase of the iterative model isn't too not quite the same as the phases of a more conventional model like the waterfall model—and consequently the procedure will take a lot of time—the magnificence of the iterative procedure is that each stage can successfully be thinned down into smaller time allotments; whatever is important to suit the requirements of the task or association [8].

Suited for Agile Associations: While a well ordered process like the waterfall model may function admirably for extensive associations with several colleagues, the iterative model truly begins to sparkle when it's in the hands of a littler, more nimble group. Especially when joined with the energy of current form control frameworks, a full “cycle process” can adequately be performed by various individual colleagues, from arranging and plan through to execution and testing, with next to zero requirement for outside input or help [9].

Easy Adaptability: An essential preferred standpoint of the iterative model is the capacity to quickly adjust to the regularly changing requirements of both the undertaking or the impulses of the customer. Indeed, even essential changes to the hidden code structure or usage [10].

Enables Risk Analysis: This is also called “Project Impact Analysis” and it is conducted at the analysis phase of the SDLC to identify irregularities in the systems requirements [11]. The Iterative model accommodates a process that identifies and assesses success factors of a project/goal.

Progress Measurement: The Iterative model also accommodates progress measurement. After each iteration, stakeholders can see a tangible system development and the development team can monitor progress after each build. This minimises project stagnation and provides a motivation for the team [12].

2.2 The Disadvantages

The major demerit of the Iterative SDLC Model is that it is only appropriate for massive system or software development project only; it is difficult to break a small programming framework into assist little serviceable augmentations/modules. The weaknesses are very common.

Costly Late-Stage Issues: While not really an issue for all activities, because of the insignificant introductory arranging before coding and usage start, while using an iterative model, it is conceivable that an unanticipated issue in plan or hidden framework design will emerge late into the task. Settling this could have conceivably destroying impacts on the time period and expenses of the task all in all, requiring a lot of future cycles just to determine one issue [13].

Increased Pressure on User Engagement: Weight on Not at all like the waterfall demonstrate, which accentuates almost all client/customer engagement inside the underlying phases of the venture amid a short time to get down to business period, the iterative

model regularly requires client engagement all through the whole of the procedure. This is now and again an appalling commitment, since each new emphasis will probably require testing and input from clients with a specific end goal to legitimately assess any vital changes [14].

Feature Creep: Not exclusively does the iterative model require client criticism all through the procedure, yet this likewise characteristically implies the task might be liable to undesired element crawl, whereby clients encounter the adjustments in every cycle, and are slanted to continually advance new demands for extra highlights to be added to future forms [1].

Management Complexity: Complexity management as a disadvantage, its methodology deals with the analysis and optimization of complexity in enterprises. Effects of complexity pertain to all business processes along the value chain and hence complexity management requires a holistic approach and due to iteration on each build or development, management will be very difficult [15].

3 Motivation for Iterative Model

The use of an SDLC often depends on several unrelated internal and external factors. For instance, iteration model is better used to smartly handle problems in the following circumstances [16]:

- When the systems requirements are clearly understood by the experts.
- Major systems requirements are initially defined even though, there is a possibility of functional enhancements from time to time.
- Unavailable resources with required skill set are scheduled for use on contract basis for specific iterations.
- The development team is just learning the new technology in use.
- Some goals and high risk features may change with time.
- There is no much time to the market constraint.

4 Practical Applications of Iterative Model

Wiki: This is the iterative design natural repository with a ‘Page History’ facility that allows prior versions to be tracked. Wiki encourages incremental modifications while living the major parts of the text unchanged [17].

Common Law: This is a principle based on legal precedent and past experience [18]. It is particular about law being an iterative design with a clear audit trail of a legal thought.

Evolution: This is where both theories of iterations and that of Natural Selection are at parallel as their processes are mere trial and error where only the most suitable design is allowed to advance and the unstable ones are disallowed from the next generation [19]. Subsequent versions of a product also get progressively better as its producers learn what works and what doesn’t in a process of refinement and continuous improvement [17]. This refinement process produces a continuous improvement in the subsequent versions of a product as the producers tend to appreciate factors that work and those that fail.

5 Requirement in Each Stage of the SDLC

The Requirement Engineering (RE) phase is most crucial to SDLC as it is a place where potential users' incomplete software needs or requirements are made complete and translated into formal specifications [13]. Since RE processes are very significant (Fig. 3) at every SDLC phase, it's thought in software development will always remain vital.

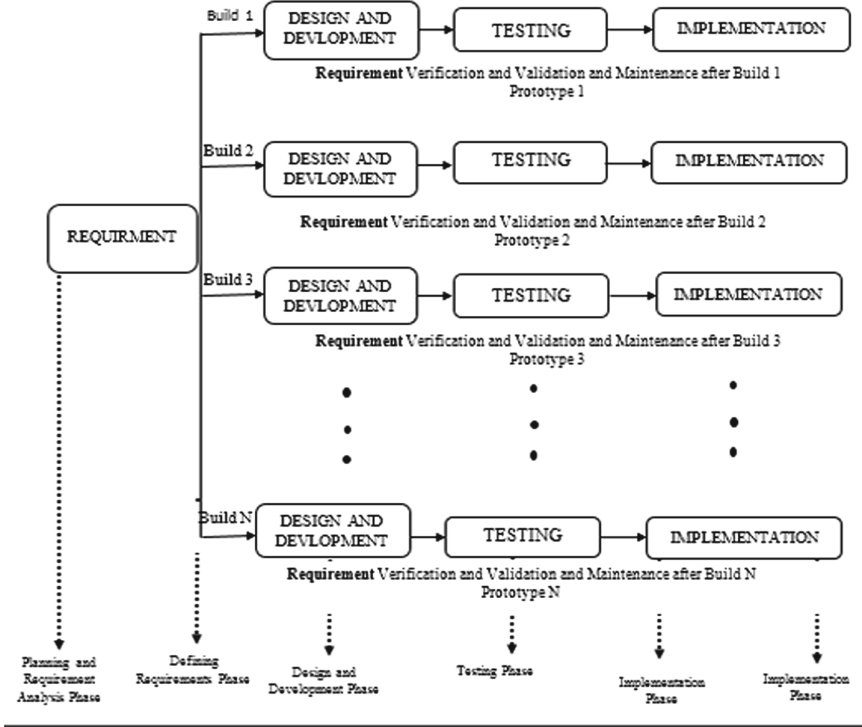


Fig. 3. Importance of requirement in the Iterative SDLC model [6]

In each iteration of Iterative model, the requirement specification document is revised to give room for system's conformity. This is the document used to build requirements at each iteration as well especially during requirement change management. Table 1 depicts some requirements against requirement activities in Iterative model, confirming that requirement activities take place at every stage of SDLC.

Table 1. Some requirements and requirements activities in Iterative model

Stage	Requirements involved	Requirement activities
Requirement	● General Stakeholder's requirements [20]	Stakeholder's requirement gathering Planning
	● Legislative requirements ● Ethical requirements [21]	Analyzing the legislative and standard requirements Deriving requirements Outlining the requirements process
	Privacy requirements [22]	Clarifying and restating the requirements
	● Space requirements ● External requirements ● Organizational Requirements [23]	Defining the requirements in a way that means the same thing to all of the stakeholders [24]
	● Standards requirements [25, 26]	Specifying the requirements according to the given standards
Design and development	● Portability requirements ● Design requirements [27]	Prioritizing the requirements. Partitioning requirements
	● Safety requirements ● Architectural requirements ● Domain requirements ● Efficiency requirements [9]	Ironing out any disagreements between customers, developers, and other stakeholders Allocating requirements
	● Delivery requirements ● Usability requirements ● Performance requirements [28, 29]	Managing requirements changes; Conducting periodic requirements reviews Validating requirements
Testing	● Compatibility requirements ● General Security requirements [30]	Testing and verifying requirements
	● Interoperability requirements ● Delivery requirements [31] ● Reliability requirements [8] ● Implementation requirements [23] ● Performance requirements	Tracking the progress Tracking requirements

6 Conclusion

The iterative SDLC can be likened to using successive approximation for software production. However, considering the mathematical methods where successive approximation is used to arrive at an answer, the significance of such methods is a function of how soon they are able to agree on a result. Requirements validation and verification in the Iterative SDLC model must therefore be done after each iteration in order to check for conformity thereby ensuring the system is on track with its main stakeholder's requirements.

References

1. Leszek Maciaszek, B.L.: Practical Software Engineering A Case-Study Approach (2004)
2. Pressman, R.S.: Software Engineering A Practitioner's Approach, 7th edn. Roger S. Pressman (2009)
3. Okesola, O.J., Okokpuji, K., Oyom, P.R.O., Kalesanwo, O., Awodele, O.: Structuring challenges in requirement engineering techniques. In: International MultiConference of Engineers and Computer Scientists, IMECS 2018, London (2018)
4. Nayab, N.: Introduction to Requirements Engineering (2009)
5. Hans-Petter, H.: Software Development (2016)
6. Motea, A.: Software development lifecycle (2015)
7. Half, R.: Basic SDLC Methodologies. Robert Half Website (2014)
8. Bayshore, S.: High Quality Software Development vs. Quick Turn-Around Software Development (2016)
9. Schwaber, K., Beedle, M.: Agile Software Development with Scrum, vol. 18 (2001)
10. Aleem, S., Capretz, L.F., Ahmed, F.: Game development software engineering process life cycle: a systematic review. *J. Softw. Eng. Res. Dev.* **4**(1), 6 (2016)
11. Peterson, G., Steven, H., Lavenhar, P.: Architectural Risk Analysis. US-Dept (2013)
12. Mahalakshmi, M., Sundararajan, M.: Traditional SDLC vs scrum methodology – a comparative study. *Int. J. Emerg. Technol. Adv. Eng.* **3**(6), 2–6 (2013)
13. Ruparelia, Nayan B.: Software development lifecycle models. *ACM SIGSOFT Softw. Eng. Notes* **35**(3), 8–13 (2010)
14. Nuseibeh, B., Easterbrook, S.: Requirements engineering: a roadmap. In: Proceedings of the Conference on Future Software Engineering - ICSE 2000, vol. 1, pp. 35–46 (2000)
15. Suryn, W.: Software Quality Engineering: A Practitioner's Approach (2014)
16. Herodotou, C., Winters, N., Kambouri, M.: An iterative, multidisciplinary approach to studying digital play motivation: The model of game motivation. *Games Cult.* **10**(3), 249–268 (2015)
17. Tanwar, D.: Software development models and their real aspects. *Int. J. Adv. Res. Sci. Eng. Technol.* **5**(3), 412–421 (2016)
18. Richard, C.: Iterative Design Paractical Application (2013)
19. Noma-Osaghae, E., Okonigene, R., Chinonso, O., Okesola, O.J., Okokpujie, K.O.: Design and implementation of an iris biometric door access control system. In: International Conference on Computational Science and Computational Intelligence (CSCI-ISPC), Las Vegas, Nevada, USA, pp. 590–593 (2017)
20. Shukla, V., Pandey, D., Shree, R.: Requirements engineering: a survey. *Requir. Eng.* **3**(5), 28–31 (2015)
21. Franch, X.: Software requirement patterns. In: Proceedings of the International Conference on Software Engineering, pp. 1499–1501 (2013)

22. Jones, C.: Software Requirements and the Ethics of Software Engineering, pp. 1–16 (2015)
23. Dvir, D., Raz, T., Shenhar, A.J.: An empirical analysis of the relationship between project planning and project success. *Int. J. Proj. Manag.* **21**(2), 89–95 (2003)
24. Okesola, J., et al.: A systematic review of requirement engineering practices in agile model. *Int. J. Mech. Eng. Technol.* **10**(02), 671–687 (2019)
25. Zarrabi, F., Mouratidis, H., Preston, D., Islam, S.: To comply software and IT system development with related laws. In: *CEUR Workshop Proceedings*, vol. 731 (2011)
26. Okesola, J.O., Okokpujie, K.O., Adebisi, A.A., Ayo, C.K.: Qualitative assessment of systematic literatures in software engineering. *J. Theor. Appl. Inf. Technol.* **96**(18), 6018–6027 (2018)
27. Mishra, A., Dubey, D.: A comparative study of different software development life cycle models in different scenarios. *Int. J. Adv. Res. Comput. Sci. Manag. Stud.* **1**(5), 2321–7782 (2013)
28. Fowler, M.: Put your process on a diet. *Softw. Dev.* **8**(12), 32 (2000)
29. Okesola, O.J., Okokpujie, K., Goddy-worlu, R., Ogunbanwo, A., Iheanetu, O.: Qualitative comparisons of elicitation techniques in requirement engineering. *ARPN J. Eng. Appl. Sci.* **14**(2), 565–570 (2019)
30. ur Rehman, T., Khan, M.N.A., Riaz, N.: Analysis of requirement engineering processes, tools/techniques and methodologies. *Int. J. Inf. Technol. Comput. Sci.* **5**(3), 40–48 (2013)
31. Powell-Morse, A.: Iterative Model: What Is It And When Should You Use It? (2016)